

Memoria Descriptiva

A Tu Manera — Ingeniería Civil / Planificación Urbana

A Tu Manera — v1.0.0

Memoria Descriptiva — A Tu Manera

1. Identificación

Campo	Valor
Nombre	A Tu Manera
Package	`com.educalab.atumanera`
Versión	1.0.0
Plataforma	Android nativo (Kotlin + Jetpack Compose)
Público objetivo	Niños y niñas de 10 a 15 años
Área	Ingeniería civil / planificación urbana
minSdk / targetSdk	24 / 34
Conectividad	Ninguna requerida (100% offline)

2. Problema y justificación

Los niños de 10-15 años rara vez tienen contacto lúdico con conceptos de planificación urbana: cómo una calle conecta un barrio, por qué un hospital necesita estar accesible, o qué significa que un presupuesto público sea limitado. **A Tu Manera** convierte estas ideas en un juego de construcción de ciudades donde cada decisión tiene una consecuencia visible y calculada en tiempo real, sin necesidad de conexión a Internet ni de recopilar ningún dato personal.

3. Objetivos

General: ofrecer una experiencia educativa entretenida sobre planificación urbana básica, con mecánicas de construcción reales (no simuladas) y progresión motivadora.

Específicos:

- Modelar de forma simplificada pero real la conectividad de una red de carreteras y la cobertura de servicios públicos.
- Traducir esas reglas en indicadores comprensibles para un niño (movilidad, servicios, áreas verdes).
- Ofrecer 30 misiones progresivas que guíen el aprendizaje sin imponer un único camino.
- Mantener toda la experiencia y los datos en el dispositivo, sin cuentas, anuncios ni analítica.

4. Público y alcance

Público: niños y niñas de 10 a 15 años, con lectura fluida de textos breves y capacidad de razonamiento espacial (cuadrículas, mapas).

Dentro del alcance:

- Construcción de 7 categorías de infraestructura sobre una cuadrícula de 10×10.

- Cálculo real de conectividad de carreteras (BFS) y cobertura de servicios por distancia de red.
- Presupuesto ficticio con validación real de gasto.
- 30 misiones evaluadas contra el estado real y persistido de la ciudad.
- Sistema de insignias (12) y monumentos coleccionables (8) desbloqueados por progreso real.
- Historial de métricas para visualizar la evolución de la ciudad.

Fuera del alcance (exclusiones explícitas):

- No es un simulador urbanístico profesional ni usa modelos de tráfico, zonificación o economía reales.
- No hay multijugador, ranking online, ni comparación entre usuarios.
- No hay compras, anuncios, cuentas ni sincronización en la nube.
- El tamaño de la cuadrícula (10×10) y el catálogo de infraestructuras (13 tipos) son deliberadamente acotados para mantener sesiones de 5-20 minutos.

5. Requisitos funcionales

#	Requisito
RF01	El sistema debe permitir construir infraestructuras de 7 categorías sobre una cuadrícula.
RF02	El sistema debe rechazar la construcción sobre una casilla ya ocupada.
RF03	El sistema debe rechazar la construcción si el presupuesto es insuficiente.
RF04	El sistema debe calcular la conectividad real de la red de carreteras mediante BFS.
RF05	El sistema debe calcular la cobertura real de educación, salud, agua y parques en función de la distancia por carretera.
RF06	El sistema debe evaluar automáticamente las 30 misiones tras cada construcción o eliminación.
RF07	El sistema debe otorgar insignias y monumentos cuando se cumplan sus condiciones reales.
RF08	El sistema debe persistir todo el progreso localmente y recuperarlo al reabrir la app.
RF09	El sistema debe permitir eliminar una construcción y recuperar parte del presupuesto.
RF10	El sistema debe mostrar un historial de métricas de la ciudad a lo largo del tiempo.

6. Requisitos no funcionales

- **Offline:** ninguna función principal depende de red. No se declara el permiso INTERNET.
- **Privacidad:** no se solicitan datos personales; el perfil usa alias y avatar local.
- **Rendimiento:** los cálculos de conectividad/coertura operan sobre una cuadrícula de 100 casillas (coste computacional trivial para BFS).
- **Accesibilidad:** contentDescription en elementos visuales relevantes, contraste de color cuidado, estados combinados con icono + texto (nunca sólo color).
- **Mantenibilidad:** separación estricta en capas domain / data / ui, con lógica de negocio 100% testeable sin Android.

7. Casos de uso principales

- **Construir infraestructura:** el jugador entra a un módulo (p. ej. "Carreteras"), elige un tipo del catálogo, toca una casilla libre → el sistema valida presupuesto y ocupación, persiste la construcción, recalcula métricas y misiones.
- **Eliminar infraestructura:** el jugador toca una construcción de su propia categoría en el módulo correspondiente → el sistema la elimina, reembolsa presupuesto, recalcula todo.
- **Completar una misión:** tras una construcción, el motor de misiones detecta que se cumplen los requisitos → otorga XP, marca la misión como completada, puede otorgar una insignia.
- **Consultar indicadores:** el jugador abre "Indicadores y colección" → ve los 6 indicadores actuales, la evolución histórica y sus insignias/monumentos.

8. Módulos y pantallas

- Onboarding (4 pantallas)
- Perfil (alias + avatar)
- Ciudad principal / Home (dashboard con cuadrícula isométrica)
- Carreteras
- Vivienda
- Educación
- Salud
- Parques
- Agua y servicios
- Transporte
- Misiones
- Indicadores y colección
- Ajustes

9. Flujo general

Onboarding → Perfil → Home (centro de experiencia) → [módulo de construcción] (7 posibles) → vuelta a Home → Misiones / Indicadores en cualquier momento. Toda acción de construcción dispara, en una sola operación de repositorio: validación → persistencia → recálculo de cobertura → recálculo de métricas → evaluación de las 30 misiones → evaluación de insignias/decoraciones.

10. Arquitectura

MVVM + Repository sobre Jetpack Compose:

domain/ → modelos puros y motor de reglas (GridEngine, MetricsCalculator,

BudgetManager, MissionEvaluator, UnlockEvaluator). Sin Android, sin Room.
data/ → entidades Room, DAOs, AppDatabase, CityRepository (orquesta domain + Room),
seeders de contenido inicial.
ui/ → ViewModels (StateFlow), pantallas Compose, componentes reutilizables,
navegación (Navigation Compose).

Ningún Composable ejecuta SQL ni contiene reglas de negocio; los ViewModels sólo exponen estado y delegan en CityRepository.

11. Datos

Ver BASE_DE_DATOS.md para el esquema completo (16 tablas), relaciones e índices. Persistencia con Room/SQLite, sin backend.

12. Reglas de negocio principales

- Una casilla sólo puede tener una infraestructura a la vez.
- No se puede construir si $\text{presupuestoGastado} + \text{coste} > \text{presupuestoTotal}$.
- La cobertura de un servicio exige que la vivienda y el servicio pertenezcan a la **misma componente conexa** de la red de carreteras y que la distancia en saltos de carretera no supere el radio de cobertura del servicio.
- Una misión se marca completa sólo cuando **todos** sus requisitos (que pueden ser de 5 tipos distintos) se cumplen simultáneamente.
- Las insignias y decoraciones se evalúan mediante condiciones explícitas y deterministas (nunca aleatorias) contra XP, misiones completadas, métricas y recuento de construcciones.

13. UX para el público objetivo

- Cuadrícula isométrica simplificada dibujada con Compose Canvas (no listas de tarjetas).
- Paleta "plano de arquitecto + ciudad viva": azul plano, ámbar sol, verde césped.
- 52 recursos vectoriales propios (iconos de infraestructura, insignias, monumentos, avatares, mascota guía, logo) — cero dependencia de imágenes externas o de Internet.
- Mascota guía ("La Grúa") en el onboarding y en el aviso de siguiente misión, sin diálogos largos.
- Feedback inmediato: banner con explicación breve al construir, rechazar o completar una misión; nunca sólo "Correcto/Incorrecto".
- Estados de módulo (bloqueado/disponible/iniciado/completado/dominado) mostrados con icono + texto + color, nunca sólo color.

14. Privacidad

Sin conexión a Internet, sin backend, sin INTERNET en el manifiesto, sin login, sin analítica ni anuncios. El perfil usa un alias libre y uno de 8 avatares locales; no se solicita nombre real, correo, teléfono ni ubicación.

15. Pruebas

57 tests JUnit (ver BUILD_REPORT.md para el detalle de ejecución real):

- 45 tests puros de dominio (GridEngine, MetricsCalculator, BudgetManager, MissionEvaluator, UnlockEvaluator) — **ejecutados realmente** con el compilador oficial de Kotlin fuera de Gradle, resultado 45/45 en verde.
- 12 tests de integración Room/Robolectric sobre CityRepository (persistencia, presupuesto, misiones, insignias, casos límite) — escritos y listos para `./gradlew testDebugUnitTest`, pendientes de ejecución en un entorno con Android SDK.

16. Limitaciones conocidas

- El entorno de construcción usado para este proyecto no tuvo acceso al repositorio Maven de Google, por lo que **no se pudo ejecutar** `./gradlew assembleDebug` ni generar un APK verificado. Ver BUILD_REPORT.md para el detalle exacto de qué se verificó y cómo.
- La simulación urbana es deliberadamente simplificada: no modela tráfico, zonificación real ni economía compleja.
- El tablero es una vista isométrica simplificada dibujada con Canvas, no un motor 3D.

17. Mejoras futuras

- Más de una ciudad por usuario (multi-partida).
- Modo "reto" con objetivos de tiempo limitado (opcional, sin penalizar por no jugar).
- Exportar/importar ciudad como archivo local para compartir sin red.
- Ampliar el catálogo de infraestructuras y monumentos.

18. Conclusiones

A Tu Manera implementa una simulación urbana simplificada pero **real**: la conectividad de carreteras se calcula con BFS genuino, la cobertura de servicios depende de la distancia real en la red, el presupuesto se valida de verdad, y las 30 misiones se evalúan contra datos persistidos, no contra valores fijos. La arquitectura MVVM + Repository con dominio puro testeable permitió verificar la lógica central de forma real durante el desarrollo, encontrando y corrigiendo tres errores concretos antes de la entrega.